



REDROB · DATA & AI CHALLENGE

Reading Between the Lines

A recruiter-grade candidate ranker that understands fit — not keywords

Hybrid retrieval + rule-based fit gating + behavioral availability + honeypot defense

Rishabh Gaikwad · 100,000-candidate pool · CPU-only, offline, < 1 min

The problem keyword filters can't solve

The brief says it plainly: recruiters miss the right person because filters match words, not meaning. This dataset weaponizes that.



The keyword trap

An HR Manager can list 9 AI skills. A pure-embedding ranker puts them at #1. The JD says this is a trap built into the data on purpose.



Honeypots

~80 profiles are subtly impossible (8 yrs at a 3-yr-old firm). Rank them in your top 10 and you signal you never read the profile.



Availability ≠ paper fit

A perfect resume that's been inactive 6 months with a 5% response rate is, for hiring, not actually available.

Right answer (from the JD): reason about the gap between what a profile says and what it means.

What the 100K pool actually looks like

I profiled the full dataset before designing anything. The shape of the pool dictates the architecture.

100K

candidate profiles

75%

India-based (25% abroad, no visa sponsorship)

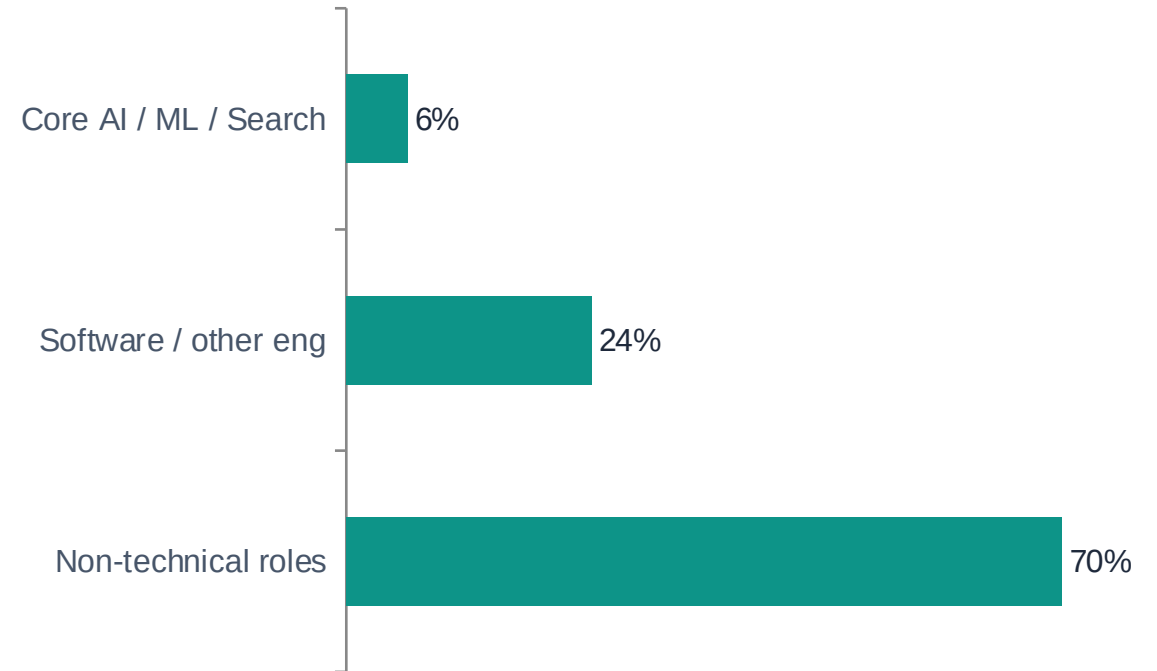
~70%

non-technical job titles

~80

honeypot profiles (DQ if >10% of top 100)

Pool by role family



The genuine target — core AI/ML engineers in India — is a thin 6% sliver. The JD agrees: "10 great matches beat 1000 maybes."

The decisive design choice

Embed what a candidate **did** (career descriptions) — never what they **listed** (the skills array, where the trap lives).



Looks perfect, scored low

Title: Marketing Manager

Skills: 9 AI keywords, all "expert"

Career text: campaigns, brand, GTM

→ no ranking / retrieval work ever shipped

Role-coherence gate kills it.



No buzzwords, scored high

Title: Backend / Platform Engineer

Skills: modest, honestly rated

Career text: built a recommendation

+ search-ranking system at a product co

Build-evidence + semantics surface it.

Dense alone is fooled by paraphrase; lexical alone is fooled by stuffing. Fusing both — then gating on role coherence — beats either.

How it works

Five explainable base components, two multipliers, one kill-switch — all CPU, deterministic, defensible line by line.



Role coherence (0.30)

Non-technical title penalized hard — the anti-trap gate.



Build evidence (0.22)

Ranking / search / rec work mined from career descriptions.



Hybrid semantics (0.20)

TF-IDF fused with bge-small dense embeddings of the narrative.



Location fit (0.18)

India required; Pune/Noida preferred; abroad down-weighted.



Experience band (0.10)

Soft curve peaking at the JD's 6–8 year ideal.



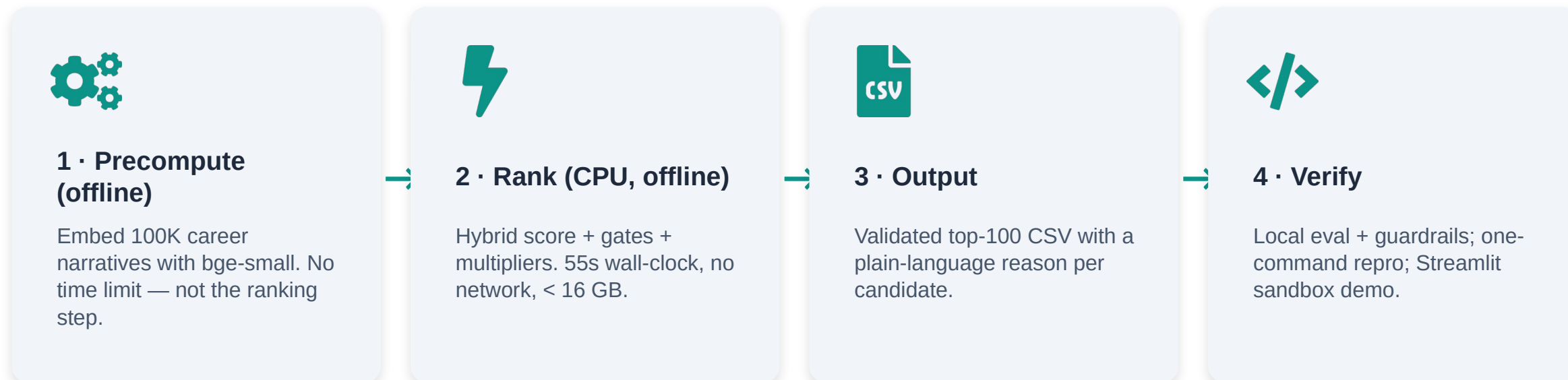
Modifiers

× disqualifiers · × availability ·
honeypot → bottom

$score = (\sum \text{weighted components}) \times \text{disqualifier-penalty} \times \text{behavioral-availability},$
honeypots forced to 0

Reasoning is assembled deterministically from real fields — specific, non-hallucinated, rank-consistent (what Stage-4 review rewards).

From candidates.jsonl to a shortlist a recruiter trusts



Reproduce in one command

```
python rank.py --candidates ./data/candidates.jsonl --out ./submission.csv
```

Meets every compute constraint: CPU-only · network off · ≤ 16 GB RAM · well under the 5-minute budget.

Results on the real pool

Versus the organizers' own sample ranking — which put HR Managers and Accountants at the top — this system avoids the trap entirely.

0

honeypots in top 100

0

non-technical roles in top 100

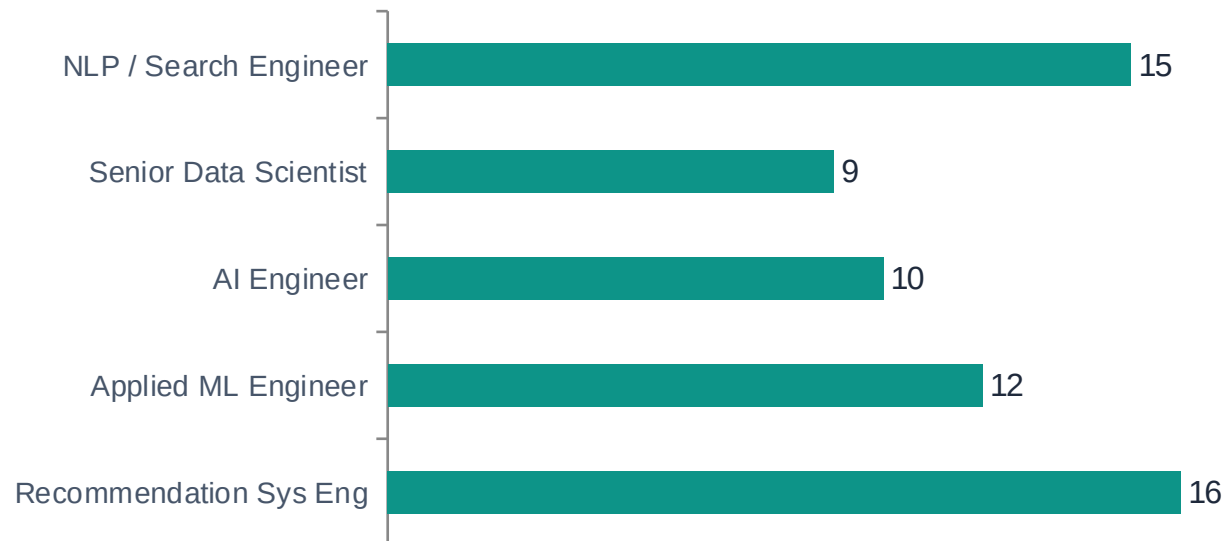
10/10

top-10 are India AI/ML engineers

55s

runtime (budget 300s)

Who the shortlist actually surfaces (top-100 role families)



Sample reasoning (rank 1)

“Senior Machine Learning Engineer with 7.2 yrs; career shows re-ranking; Pune/Noida-based; responsive to recruiters (61%).”

Specific · grounded in real fields · matches the rank.

Built to survive every stage

The composite weights NDCG@10 at 50% — so the system is engineered to be ruthless and precise at the very top.



Stage 3 — reproduction

One command, CPU-only, no network, 55s. Honeypot rate 0%.



Stage 4 — review

Modular, documented code; specific non-hallucinated reasoning; real git iteration.



Stage 5 — defense

Every weight and gate is hand-encoded from the JD and explainable in plain terms.

Not an AI-keyword counter. A system that reads profiles the way a great recruiter does.